

adapting a real app for

IPHONE DUO

EPISODE 2: THE FOLD

Every code snippet from the video, before and after, in order.

by Craig Clayton

HOW TO READ THIS

BEFORE the code as it was in Countdown2Binge.

AFTER the code after the change.

NEW code that did not exist before.

Snippets that pair up say so, for example "Pairs with 01."

Reserved regions and the hinge need iOS 27.1. Snippet 04 shows how to guard the modifier for iOS 26.

Compiled against the Xcode 27.1 RC SDK. Snippets 03, 04 and 05 are the code that ships in the app.

01 The Notification Settings Overlay

BEFORE

NotificationOnboardingSheet.swift, NotificationSettingsOverlay. Two full-screen backdrops and a centered card.

NOTE

Nothing here knows a fold exists. On iPhone Duo, partially open, the centered card lands right on the fold and the Save button runs across the crease.

```
var body: some View {
    ZStack {
        // Blurred dimmed background
        Rectangle()
            .fill(.ultraThinMaterial)
            .ignoresSafeArea()

        Color.black.opacity(0.80)
            .ignoresSafeArea()
            .onTapGesture { onDismiss() }

        // Centered card
        VStack(spacing: 0) {
            // header, options, save button
        }
        .background(Color(hex: "#18181b"))
        .clipShape(RoundedRectangle(cornerRadius: 20))
        .padding(.horizontal, 24)
    }
    .onAppear {
        settings = settingsStore.settings
    }
}
```

02 Asking Where the Fold Is

NEW

A geometry proxy gives you the reserved regions. Asking only for divisions returns just the fold.

```
proxy.reservedRegions(kind: .division, options: .includeInactive)
```

03 The Fold Avoidance Modifier

NEW

Utilities/Extension+FoldAvoidance.swift. Needs iOS 27.1.

NOTE

The first closure reads the right edge of the active fold, or zero if there isn't one. The second stores it, and the view is padded by that amount from the leading edge. The measuring happens on a clear background layer, not on the view being padded.

```
@available(iOS 27.1, *)
private struct FoldAvoidanceModifier: ViewModifier {
    @State private var foldMaxX: CGFloat = 0

    func body(content: Content) -> some View {
        content
            .padding(.leading, foldMaxX)
            .background {
                Color.clear
                    .onGeometryChange(for: CGFloat.self) { proxy in
                        proxy.reservedRegions(kind: .division, options: .includeInactive)
                            .first(where: { $0.isActive })?.frame.maxX ?? 0
                    } action: { maxX in
                        foldMaxX = maxX
                    }
            }
    }
}
```

04 The iOS 26 Wrapper NEW

Utilities/Extension+FoldAvoidance.swift, below the modifier.

NOTE

Check availability once, here, and call `avoidingActiveFold()` everywhere else. On iOS 27.1 and later the modifier applies. Below that, the view is returned unchanged.

```
extension View {  
    /// Keeps this view on the trailing side of an active fold, on iOS 27.1 and later.  
    @ViewBuilder  
    func avoidingActiveFold() -> some View {  
        if #available(iOS 27.1, *) {  
            modifier(FoldAvoidanceModifier())  
        } else {  
            self  
        }  
    }  
}
```

05 The Overlay, Card Only

AFTER

NotificationOnboardingSheet.swift, NotificationSettingsOverlay. Pairs with 01.

NOTE

The card is wrapped in a container that fills the space, and the modifier goes on that container. The two backdrops stay full screen, and tapping the left side still dismisses the overlay.

```
var body: some View {
    ZStack {
        // Blurred dimmed background
        Rectangle()
            .fill(.ultraThinMaterial)
            .ignoresSafeArea()

        Color.black.opacity(0.80)
            .ignoresSafeArea()
            .onTapGesture { onDismiss() }

        // Centered card, in a container that fills the space so the fold
        // modifier insets only the card and leaves the backdrops full-screen.
        ZStack {
            VStack(spacing: 0) {
                // header, options, save button
            }
            .background(Color(hex: "#18181b"))
            .clipShape(RoundedRectangle(cornerRadius: 20))
            .padding(.horizontal, 24)
        }
        .frame(maxWidth: .infinity, maxHeight: .infinity)
        .avoidingActiveFold()
    }
    .onAppear {
        settings = settingsStore.settings
    }
}
```

06 Reading the Hinge

NEW

HingeReadout.swift. A standalone example. Countdown2Binge doesn't use the hinge. Needs iOS 27.1.

NOTE

The hinge is optional, so the status and the angle can both come back empty. The readout only shows when it has both. Otherwise it shows No hinge.

```
@available(iOS 27.1, *)
struct HingeReadout: View {
    @State private var status: DeviceHinge.Status? = nil
    @State private var angle: Angle? = nil

    var body: some View {
        VStack {
            if let status, let angle {
                Text("\(angle.degrees, specifier: "%.0f") degrees")
                Text(status == .closed ? "closed"
                    : status == .partiallyOpen ? "partially open"
                    : "fully open")
            } else {
                Text("No hinge")
            }
        }
        .onHingeChange { _, newContext in
            status = newContext.hinge?.status
            angle = newContext.hinge?.angle
        }
    }
}
```